Matplotlib

Fernando Nemec

O que é matplotlib?

O **Matplotlib** é a principal biblioteca de visualização de dados do Python. Ele permite transformar números brutos em **gráficos claros e informativos**, ajudando a identificar padrões e tendências.

Com o Matplotlib, é possível criar **gráficos de linhas, barras, histogramas, scatter plots e muito mais**, além de personalizá-los para uma apresentação profissional. É amplamente usado em **ciência de dados, engenharia, finanças e pesquisa** para tornar a análise de dados mais visual e intuitiva.

Resumindo: Matplotlib é a ferramenta que transforma dados em **insights visuais!**

Exemplo 1: Gráfico de linha simples

```
import matplotlib.pyplot as plt

# Criando um gráfico de linha simples
x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 40]

plt.plot(x, y)
plt.xlabel('Eixo X')
plt.ylabel('Eixo Y')
plt.title('Meu Primeiro Gráfico')
plt.show()
```

Exemplo 2: Scatter / Dispersão

```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.rand(50)
y = np.random.rand(50)

plt.scatter(x, y, color='blue')
plt.xlabel("Variável X")
plt.ylabel("Variável Y")
plt.title("Exemplo de Scatter Plot")
plt.show()
```

Exemplo 3: Gráfico de barras

```
import matplotlib.pyplot as plt
import numpy as np

categorias = ['A', 'B', 'C', 'D']
valores = [4, 7, 1, 8]

plt.bar(categorias, valores)
plt.show()
```

Exemplo 4: Histograma

```
import numpy as np
dados = np.random.randn(1000)

plt.hist(dados, bins=20, edgecolor='black')
plt.show()
```

Exemplo 5: Boxplot

```
dados = [7, 8, 9, 10, 11, 12, 50]  # Outlier proposital

plt.boxplot(dados)
plt.show()
```

Exemplo 6: Heatmap / Mapa de calor

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Criando dados: 7 dias da semana x 6 períodos do dia
dias = ["Seg", "Ter", "Qua", "Qui", "Sex", "Sáb", "Dom"]
horas = ["Manhã", "Tarde", "Noite", "Madrugada", "Almoço", "Happy Hour"]
vendas = np.random.randint(0, 50, (7, 6))  # Gera valores aleatórios de vendas

# Criando o heatmap
plt.figure(figsize=(8, 5))
sns.heatmap(vendas, annot=True, cmap="coolwarm", xticklabels=horas, yticklabels=dias)

plt.title("Heatmap de Vendas por Dia e Período")
plt.xlabel("Período do Dia")
plt.ylabel("Dia da Semana")
plt.show()
```

Exemplo 7: Personalizando Gráficos

```
plt.plot(x, y, color='red', linestyle='--', marker='s', label='Série de Dados')
plt.xlabel('Eixo X')
plt.ylabel('Eixo Y')
plt.legend()
plt.grid(True)
plt.show()
```

Exemplo 8: Múltiplos Gráficos na Mesma Figura

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))

ax[0].plot(x, y, color='blue')
ax[1].bar(categorias, valores, color='green')

plt.show()
```

Visualizar dados não é só fazer um gráfico bonito

- **A visualização deve responder a uma pergunta, não só mostrar dados.**
- O gráfico certo **destaca padrões e relações importantes**.
- O gráfico errado pode **induzir a interpretações erradas**.

Bom Gráfico, Mau Planejamento...

```
import numpy as np
import matplotlib.pyplot as plt

ids = np.arange(1, 21)  # IDs de 1 a 20
valores = np.random.randint(50, 100, 20)

plt.bar(ids, valores)  # ISSO É INÚTIL! ID NÃO É UMA CATEGORIA RELEVANTE!
plt.show()
```

Bom Gráfico, Bom Planejamento...

```
plt.hist(valores, bins=5, edgecolor='black')  # Agora faz sentido!
plt.show()
```

Moral de história

Escolher o gráfico errado pode matar sua análise antes mesmo dela começar.

Erros comuns que prejudicam a interpretação

Erro 1: Usar gráficos errados para o tipo de dado

❌ **Gráfico de pizza para muitas categorias** → Se há **muitas fatias**, fica impossível interpretar.

❌ **Gráfico de linhas para dados categóricos** → **Linha sugere continuidade**, então não faz sentido para categorias isoladas.

❌ **Gráfico de barras para dados numéricos contínuos** → Para isso, usa-se um **histograma**.

Erros comuns que prejudicam a interpretação

Erro 2: Manipular escalas para distorcer a informação

❌ **Eixo Y truncado para exagerar variações** → Faz uma diferença pequena parecer enorme.

❌ **Escalas inconsistentes entre gráficos comparados** → Comparação desleal.

Erros comuns que prejudicam a interpretação

Erro 3: Gráficos que não informam nada útil

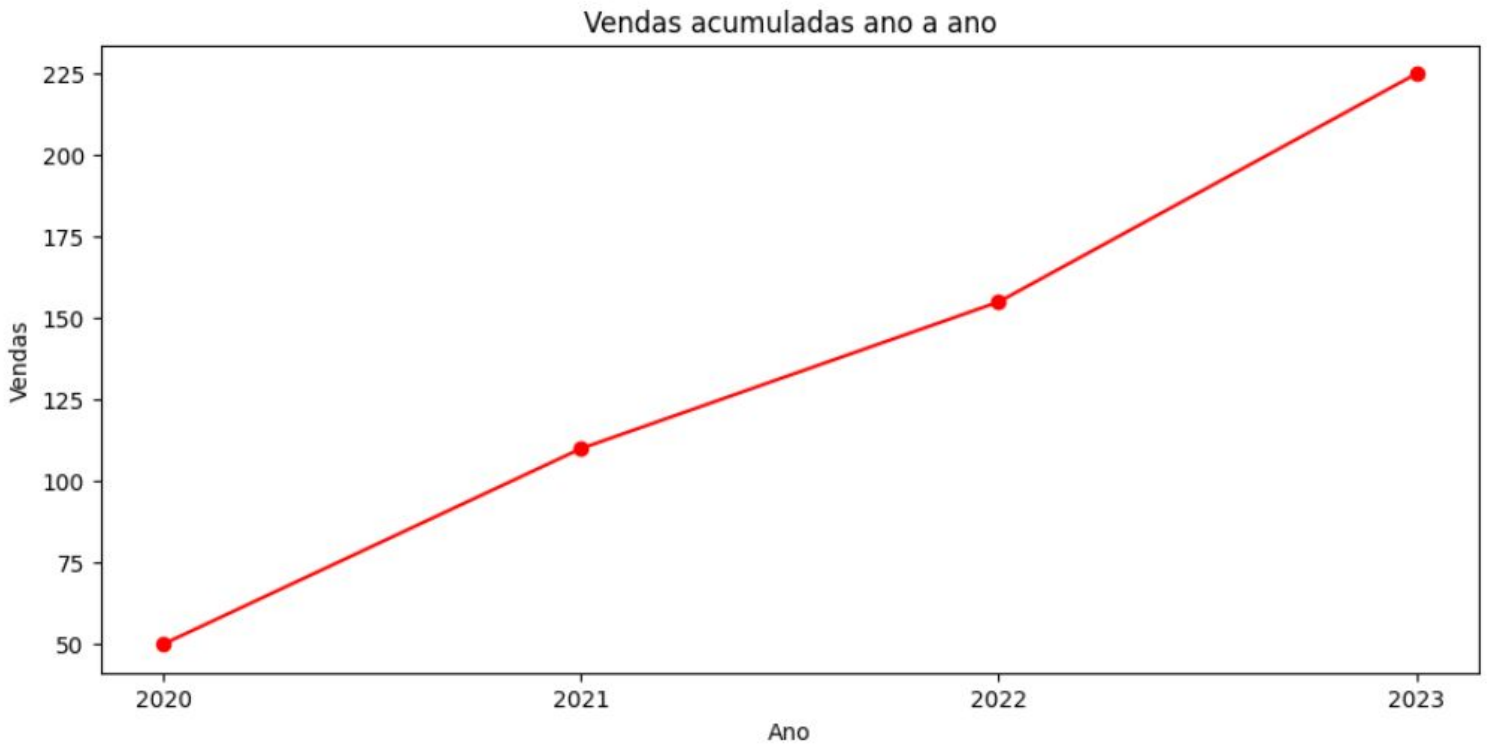
❌ **Visualizações desnecessárias** → Um gráfico **não pode ser uma tabela mais feia.**

❌ **Falta de rótulos e legendas** → Ninguém entende nada sem contexto.

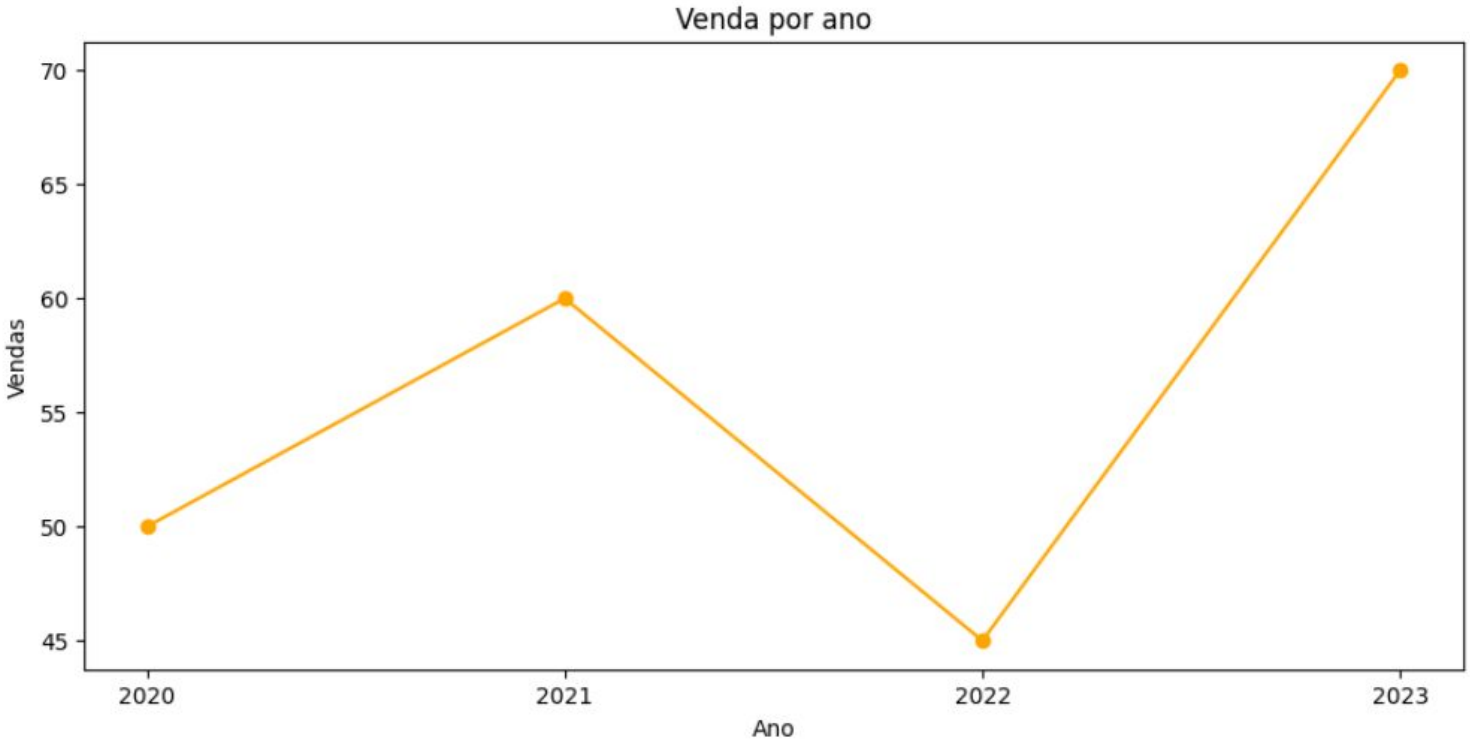
Comparação Gráfico Ruim x Gráfico Bom

```
import numpy as np
import matplotlib.pyplot as plt

# Criando dados
anos = ["2020", "2021", "2022", "2023"]
valores = [100, 105, 107, 110]
fig, ax = plt.subplots(1, 2, figsize=(10, 4))

# Gráfico manipulador (eixo truncado)
ax[0].bar(anos, valores, color='red')
ax[0].set_ylim(100, 110)  # Eixo Y manipulado
ax[0].set_title("Gráfico Manipulado")

# Gráfico correto (eixo completo)
ax[1].bar(anos, valores, color='green')
ax[1].set_ylim(0, 120)  # Eixo Y adequado
ax[1].set_title("Gráfico Correto")
plt.show()
```

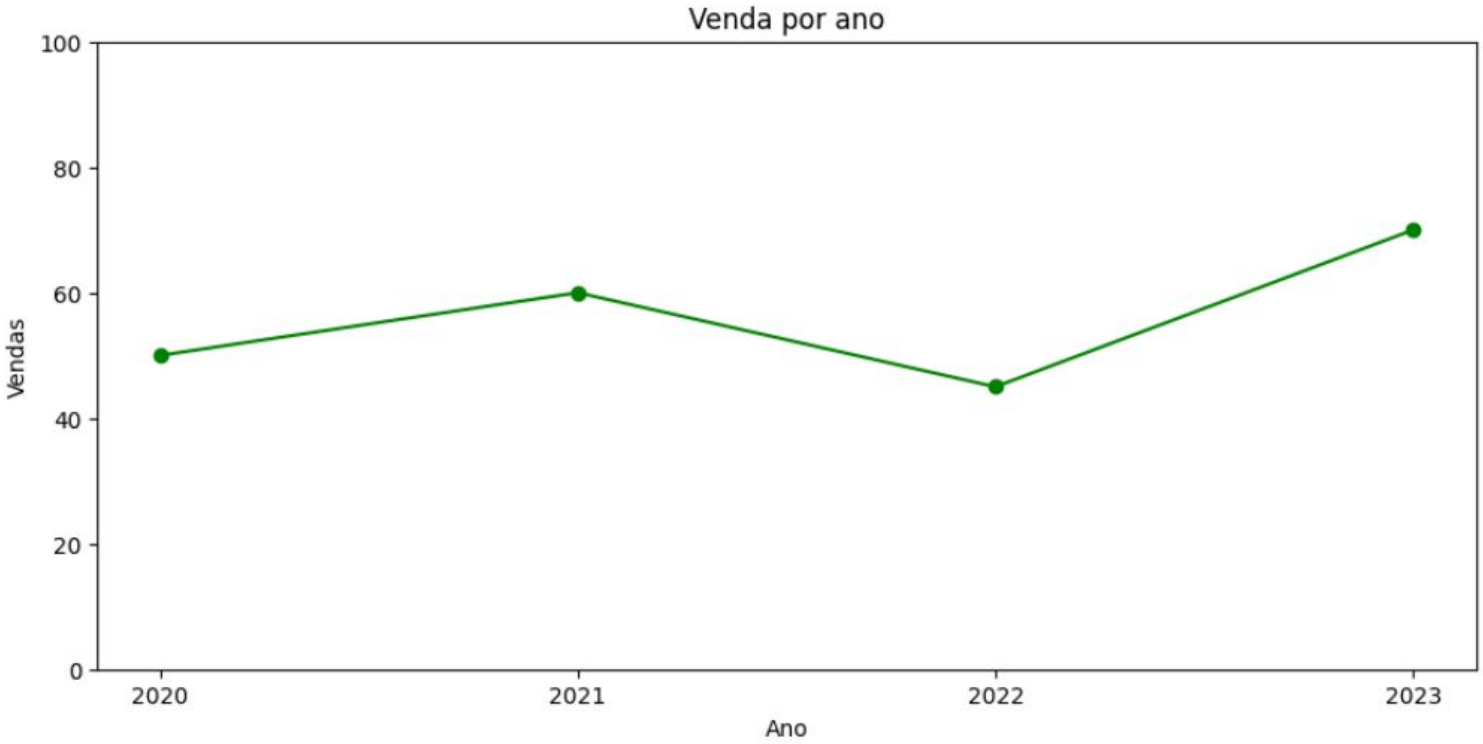
Truquezinho sujo... :)



Agora corrigido... mas dá pra melhorar!



Agora beeeem melhor!



Problemas identificados

1. **Eixo X não representa o tempo corretamente** → Ele usa **vendas acumuladas** como se fossem anos.
2. **Distorce a percepção de crescimento** → Parece que as vendas **só aumentam**, mas na verdade **caíram em 2022**.
3. **Leva a decisões erradas** → Se um gestor olhar esse gráfico errado, pode achar que **o crescimento foi constante**, quando na verdade **não foi**.

Micro Desafio

1. Montagem de base de dados: montar planilha com as partidas jogadas pelo Brasil na Copa de 2022, incluindo as eliminatórias. Armazenar a quantidade de gols do Brasil, a quantidade de gols do adversário e o nome do adversário.
2. Converter a planilha para um arquivo csv (arquivo separado por vírgula).
3. Usar Pandas carregar o arquivo. Testar se a carga ocorreu corretamente.
4. Criar algoritmo para calcular a média de gols do Brasil por partida.
5. Criar um gráfico de linha simples com os gols do Brasil por partida.
6. Acrescentar ao mesmo gráfico uma reta indicando a média de gols do Brasil. Analisando o gráfico, em quantas partidas o Brasil marcou um número de gols acima da média?
7. Criar um gráfico de barras com os adversários do Brasil no eixo x e os gols marcados no eixo y.

A Batalha de Gráficos: Quem Conta a Melhor História?

Regras do jogo:

1. A turma será dividida em grupos.
2. Cada par de grupos receberá o mesmo conjunto de dados.
3. Cada grupo precisa criar 3 gráficos OU MAIS que melhor representem os dados recebidos.
4. Os gráficos serão apresentados para a turma, que votará no conjunto que contou a melhor história.

A Batalha de Gráficos: Quem Conta a Melhor História?

O desafio não é apenas fazer gráficos bonitos, mas sim **escolher as visualizações mais eficazes** para transmitir informação de qualidade.

Que comecem os trabalhos... e que vença o melhor cientista de dados!

A Batalha de Gráficos: Quem Conta a Melhor História?

Regras da Batalha de Gráficos

Quem começa?
Um grupo será **sorteado aleatoriamente** para iniciar a batalha. Esse grupo será o **desafiante**.

Escolhendo o oponente:
O desafiante tem o direito de **escolher qualquer outro grupo da sala** para ser o **desafiado**.

Escolhendo o campo de batalha:
O grupo **desafiado** terá a vantagem de **escolher o dataset** que será utilizado no duelo.

Quem conseguirá criar a melhor história visual e convencer a turma? Que comecem os desafios!